



A Reinforcement Learning Guided Large Neighborhood Search for the Dynamic Electric Autonomous Dial-a-Ride Problem

Laurenz Tomandl¹(✉), Maria Bresich¹, Günther R. Raidl¹, Yi Mei²,
Steffen Limmer³, and Tobias Rodemann³

¹ Algorithms and Complexity Group, TU Wien, Vienna, Austria
{ltomandl,mbresich,raidl}@ac.tuwien.ac.at

² Victoria University of Wellington, Wellington, New Zealand
yi.mei@vuw.ac.nz

³ Honda Research Institute Europe, Offenbach, Germany
{steffen.limmer,tobias.rodemann}@honda-ri.de

Abstract. We consider a version of the Dynamic Electric Autonomous Dial-a-Ride Problem (DynEADARP) recently proposed and approached by means of a Genetic Programming (GP) hyperheuristic. This problem integrates the challenges of the static dial-a-ride problem with those of considering the charging of the fleet of electric vehicles and the dynamic nature of customer requests, i.e., the online aspect. The objective is to minimize the total travel time for serving all requests in a given time horizon together with penalties for late request pickups. We consider an optimization-based solution framework that utilizes a Large Neighborhood Search (LNS) for the static variant of the problem. This LNS is (re-)applied whenever new requests arrive and always considers the current state of the vehicles and all available, not yet served requests. Analyzing results of a first version in which the LNS performs in a pure myopic way that does not consider expected future requests, we observe two major weaknesses: (a) charging is often done much too late, and (b) it is not always good to head to planned pickup locations as early as possible. To address these weaknesses, we utilize reinforcement learning (RL) for learning two functions offline that are used within the LNS to incentivize earlier charging and waiting dependent on features of the current state and the expected number of future requests in a sensible way. An experimental evaluation shows that (a) the LNS-based approach can scale well to large instances with up to 10000 requests, (b) it outperforms the former GP hyperheuristic with a gap of up to 100%, and (c) the RL-guided LNS substantially improves upon the myopic LNS on instances up to 20 times larger than the training instances.

Keywords: Large Neighborhood Search · Reinforcement Learning · Dynamic Electric Autonomous Dial-a-Ride Problem

1 Introduction

The demand for efficient personal logistics solutions remains a persistent challenge in modern transportation systems, encompassing services like private car usage, taxis, and dial-a-ride. The latter involves shared transportation for multiple customers with different pickup and drop-off locations served by a single vehicle and offers cost and environmental benefits over traditional taxis. However, they often result in longer travel and waiting times due to reduced flexibility. Effective route and stop planning is essential to mitigate these drawbacks, leading to the formulation of the Dial-a-Ride Problem (DARP) by Cordeau and Laporte [8]. The DARP aims to minimize routing costs while adhering to constraints such as time windows, vehicle capacity, maximum ride durations, and route duration limits. Over time, the DARP has evolved, with adaptations addressing specific contexts, as reviewed by Molenbruch et al. [20], Ho et al. [14], and Cordeau and Laporte [6].

A significant adaptation of the DARP involves integrating electric and autonomous vehicles. Autonomous vehicles can operate continuously without driver breaks, enabling uninterrupted service. Electric Vehicles (EVs), however, introduce challenges such as reliance on possibly underdeveloped charging infrastructure and lengthy recharging times. These factors necessitate careful route planning and charging station management. Another adaptation arises from the dynamic nature of real-world transportation systems, where customer requests arrive over time often with short lead times. This extends the DARP to the dynamic dial-a-ride problem, as described by Ho et al. [14]. When combined with electric and autonomous vehicle challenges, the problem becomes the Dynamic Electric Autonomous Dial-a-Ride Problem (DynEADARP).

Solution approaches for DARP variants are broadly categorized into exact methods and (meta-)heuristic ones. Exact methods, such as Mixed Integer Linear Programming (MILP), guarantee optimal solutions but suffer from scalability issues, making them suitable only for small, static problem instances. In contrast, heuristic and metaheuristic approaches, including Large Neighborhood Search (LNS), genetic algorithms, deterministic annealing, and tabu search, are more scalable and adaptable to large, dynamic scenarios.

In this work, we propose a novel approach to the DynEADARP, building on earlier work on an LNS for the static Electric Autonomous Dial-A-Ride Problem (EADARP) [5] inspired by other deep learning supported heuristics such as [33]. In the DynEADARP, customer requests arrive dynamically, with only approximate expected arrival rates known in advance. To address this problem, we start in a classical optimization-based myopic fashion, iteratively solving the static problem variant at a given time while considering known unhandled requests. When new requests arrive, we adapt to the new knowledge and re-optimize using an LNS similar to Bresich et al. [5]. Our LNS handles charging stops by inserting them on-the-fly using exact dynamic programming. While this myopic approach performs well in some cases, it has two major weaknesses: (a) charging is often done much too late with vehicles waiting until the latest possible moment to charge their battery, and (b) heading to pickup locations as

early as possible can be suboptimal (see Sect. 4.5). We address the first issue by a heuristic extension of the objective function targeted by the dynamic program for the charging stop insertion. While this simple extension already helps, we further improve the solution quality by applying Reinforcement Learning (RL) offline to learn a policy more appropriately incentivizing charging and also making better decisions with respect to waiting times within routes. In this way, we also make effective use of known expected future request arrival rates.

We conduct computational experiments using randomly generated instances of varying sizes and arrival rates based on real-world scenarios. Our methods are compared to a recent genetic programming hyperheuristic [15]. Results show that our LNS-based approaches outperform the latter substantially. In contrast to the genetic programming hyperheuristic, the LNS-based methods scale well to instances with up to 10000 requests. The LNS with the heuristic extension and in particular the RL-guided LNS further improve solution quality on most instances substantially over the myopic LNS.

The remainder of this paper is structured as follows. The next section reviews related work. Section 3 then introduces the considered DynEADARP in detail. In Sect. 4, we dive into the details of the proposed LNS-based approach, explaining its design and enhancements. The offline RL for obtaining a policy to better guide the LNS is described in Sect. 5. Section 6 presents and discusses the results of computational experiments. Finally, we conclude in Sect. 7 with a summary of our findings and point out potential avenues for future research.

2 Related Work

Research on DARP and its variants has predominantly focused on static problem formulations, with only limited attention given to dynamic variants such as the DynEADARP. Below, we review relevant literature on the DynEADARP, its static counterpart (EADARP), and related dynamic DARP formulations. A DynEADARP variant was first introduced by Bongiovanni et al. [3], who proposed a two-phase metaheuristic. The first phase employs an efficient insertion heuristic to generate initial solutions, while the second phase optimizes these solutions using local search with intra- and inter-route swapping operations. This approach is event-driven, responding to newly arriving requests in real time. In subsequent work, Bongiovanni et al. [2] enhanced their local search with a Machine Learning-guided LNS, utilizing an offline-trained Random Forest model to select destroy and repair combinations. Kullman et al. [16] adopted a deep reinforcement learning approach, approximating Q -values to derive a policy for scheduling stops on routes. Huang et al. [15] explored a genetic programming hyperheuristic for the DynEADARP, learning rule-based policies to decide actions such as waiting, charging, or serving customers. We remark that most of these existing works on the DynEADARP address different variants of the core problem, making it hardly possible to compare them directly. For example, while [2] considers as a minimization goal the total travel duration plus the total extra ride time of users in comparison to direct connections, [15] considers the total travel duration together with the total lateness of users. Moreover,

some works require all requests to be served, while others include the decision making of which requests should be accepted and scheduled. We will build upon the latest DynEADARP variant as proposed in [15].

The static EADARP was introduced by Bongiovanni et al. [4] and has seen growing interest in recent years. In this original work, exact MILP approaches were presented, which, however, are only applicable to small instances. In a follow-up work, Bongiovanni et al. [1] proposed a scheduling heuristic based on linear programming theory, which is applicable to classical DARP variants but is further combined with a charging insertion heuristic. Su et al. [31] developed a deterministic annealing heuristic for the EADARP, while Limmer [17] introduced a bi-level LNS that separates charging station insertion and route optimization into distinct phases. Lastly, Bresich et al. [5] proposed an LNS-based algorithm with a highly effective on-the-fly charging station insertion mechanism; this approach is currently state-of-the-art for the static EADARP, and we adopt it with some changes for our DynEADARP approach.

The classic DARP, introduced by Cordeau and Laporte [8], has seen numerous solution approaches. Exact methods include Mixed Integer Linear Programming with Branch-and-Cut [7, 28] and Branch-and-Price [11], as well as dynamic programming approaches [27]. Heuristic and metaheuristic methods include among others tabu search [8], genetic algorithms [10, 25], and hybrid metaheuristics combining dynamic programming with other techniques [27]. Comprehensive reviews of these methods can be found in [7, 14, 21].

For dynamic DARP variants, Coslovich et al. [9] and Tafreshian et al. [32] independently proposed two-phase methods, combining offline learning phases with an online request assignment phase. Van Lon et al. [34] described an evolutionary approach for the synthesis of a multi-agent system. Sayarshad and Gao [29] formulated the dynamic DARP as a Markov Decision Process (MDP), using a multi-server queue policy to maximize social welfare and minimize passenger travel time. Gaul et al. [12] employed an event-based MILP within a rolling horizon framework, solving most instances with up to 500 requests optimally. Liu et al. [18] addressed large-scale instances with 300,000 requests using a guided insertion process informed by historical data. Employing reinforcement learning, Heitmann et al. [13] formulated an MDP to approximate a value function and optimize customer travel fees. Beyond DARP, dynamic routing problems such as the dynamic vehicle routing problem and the dynamic capacitated arc routing problem have also been studied. Among others, Pillac et al. [23] provide insights into solution approaches for these problems, which share similarities with the DynEADARP in terms of real-time decision-making and resource constraints.

3 Dynamic Electric Autonomous Dial-a-Ride Problem

We now define the DynEADARP following the variant of [15]. The problem is defined on a complete directed graph $G = (V, A)$ with the node set V representing all relevant locations, including pickup and drop-off points for customer requests, charging stations, and the initial locations of vehicles. The set of

directed arcs $A = \{(u, v) : u, v \in V, u \neq v\}$ represents the fastest routes between pairs of nodes. For each arc $(u, v) \in A$, we know its travel duration $d_{u,v}$ and battery consumption $b_{u,v}$. We assume that the triangle inequality holds for both. Expected service durations for picking up or dropping off customers are assumed to be included in the travel durations.

There are n requests $R = \{1, \dots, n\}$, and each request $i \in R$ is characterized by its arrival time t_i^{arr} (i.e., when the request becomes available for planning), its pickup and drop-off locations $v_i^{\uparrow}, v_i^{\downarrow} \in V$, respectively, a pickup time window $[t_i^{\text{start}}, t_i^{\text{end}}]$, a load value l_i (typically corresponding to the number of passengers), and a maximum ride time duration d_i^{max} that must not be exceeded.

These requests are to be served by a fleet K of n_K homogeneous electric autonomous vehicles. Each vehicle $k \in K$ shares the same maximum passenger load C and battery capacity Q . A vehicle has an associated initial charge level $B_{k,0}$ and an initial location $v_k^{\text{ini}} \in V$.

Moreover, there are n_S charging stations $S \subset V$. These are uncapacitated, time window independent and have a homogeneous charging rate α . When an EV comes to such a station, we assume it will always charge until its battery is full; for an EV k with current battery level $B_{k,i}$ the charging duration is calculated as $(Q - B_{k,i})/\alpha$. Charging is only allowed when no customers are on board.

A DynEADARP solution consists of a route for every vehicle $k \in K$, where a route comprises a sequence of request pickup and drop-off stops, along with charging stops. Every pickup and charging stop is associated with an arrival time t_i . Each vehicle starts from its initial location v_k^{ini} at time 0. Routes must be feasible in the sense that the vehicle never runs out of battery charge, the maximum load C is never exceeded, and that for each served request i , the pickup location is visited the earliest at time t_i^{start} and the drop-off location later, with the maximum ride duration d_i^{max} being respected. Each request must be served by exactly one vehicle. The pickup time window's end only is a soft constraint to guarantee feasibility. The objective is to minimize a weighted sum of the total travel time and the lateness of customer pickups

$$\sum_{k \in K} d_k^{\text{travel}} + \rho \cdot \sum_{i \in R} d_i^{\text{late}} \quad \text{with } d_i^{\text{late}} = \max(t_i - t_i^{\text{end}}, 0), \quad (1)$$

where d_k^{travel} represents the total driving duration of vehicle k , which does not include waiting and charging times, and d_i^{late} the lateness of each request's pickup, which is weighted by a penalty factor ρ .

4 Proposed Methods

We start by first describing the general optimization based dynamic framework we apply to solve the DynEADARP. The subproblems arising for each state of the dynamic process are solved by an LNS, which is explained next. Within the LNS, candidate solutions are efficiently evaluated by first breaking them down into so-called fragments, described in Sect. 4.3, and then inserting charging stops in an optimized fashion as described in Sect. 4.4, before finally calculating the objective value.

Algorithm 1. Dynamic Framework

Input: DynEADARP instance *inst* to be solved
Output: complete solution *sol* serving all requests

```

1: sol  $\leftarrow$  empty solution
2: finished  $\leftarrow$  False
3: t  $\leftarrow$  0
4: state  $\leftarrow$  subproblem(sol, inst, t)
5: while not finished do
6:   subsol  $\leftarrow$  LNS(state, inst)
7:   t  $\leftarrow$  getNextRequestArrivalTime(inst, t)
8:   finished  $\leftarrow$  update(sol, subsol, state, t)
9: end while
10: return sol

```

4.1 Dynamic Framework

We propose a dynamic framework to handle the DynEADARP, which continuously adapts to incoming customer requests in (simulated) real time. The framework operates in a rolling horizon manner, re-solving the static problem variant with so far available knowledge as new requests arrive. Algorithm 1 shows this framework in pseudo-code. The framework constructs a static subproblem instance (Line 4) considering the current state, i.e., yet unserved requests up to the current time t and the current locations, battery levels, and availability times t^{avail} of the vehicles. At each iteration, the LNS algorithm (Sect. 4.2) is applied to this static subproblem to generate a preliminary solution *subsol*. This solution is realized until new knowledge is obtained by one or more newly arriving requests at some time t (Line 7). In our simulation, this realization is performed by updating the overall solution *sol* by transferring fragments from *subsol* that started before time t (Line 8) and updating the state by removing requests that are now served and adding new requests that are now available at time t . These fragments are immutable, and their requests are removed from the set of unserved requests. This process is repeated until all requests are served, and solution *sol* is finally returned.

4.2 Large Neighborhood Search

Our LNS follows a classical scheme similar to [24] as it has already been successfully applied to many variants of vehicle routing problems. The main principle is that an incumbent solution is iteratively “destroyed”—in our case by removing randomly selected requests with their pickup and drop-off stops from the vehicle routes—and then “repaired” by reinserting the removed requests into the routes according to a certain strategy. When the obtained new solution is better than the original one, it is accepted as a new incumbent, otherwise discarded.

More specifically, we follow here the LNS for the static EADARP from [5] to a large extent and therefore refer to this work for details. While we only consider

a simple uniform random destroy operator that is parameterized by the number of requests to be removed, we apply three different repair strategies:

Greedy repair: It considers all possible insertion positions of all removed requests and inserts the single one that increases the objective value the least. This process is repeated until all unserved requests are reinserted.

Order repair: Removed requests are sorted in non-decreasing order of their time window starting times t_i^{start} , and are then inserted in this order in a way that increases the objective value the least.

Random repair: The removed requests are inserted in random order, with each one placed in a way that increases the objective value the least.

The initial solution for the LNS is generated by applying the greedy repair to an initially empty solution. As this procedure is rather slow for larger instances, we use the faster order repair when there are more than 100 requests.

A crucial aspect is the way we deal with charging stops: The LNS does not directly consider these but only works on sequences of pickup and drop-off stops. Only when it comes to the evaluation of a candidate solution for obtaining its objective value, charging stops are inserted On-The-Fly (OTF) in the route of each vehicle in an optimal fashion. Before going into detail in Sect. 4.4, we have to consider an important prerequisite in the following section.

4.3 Fragments

We adopt the concept of *fragments*, originally introduced by Rist and Forbes [26] and also applied in [5]. A fragment is defined as a closed sequence of stops that starts when a customer enters an empty vehicle and ends when the last customer on board leaves the vehicle, making it empty again. The advantage of considering such fragments is that the relative timing for the stops within such a fragment can be optimized independently of the rest of a route. When this relative timing information is stored with all fragments, it is later on in case of modifications of a route only necessary to recalculate the times anew for the fragments that have changed.

In [5, 31], fragments were independently solved by means of a linear program. Due to the different objective function, the situation is simpler in our case when restricting candidate solutions by not allowing any waiting within fragments, i.e., when having passengers on board. Although there can be cases where this kind of waiting might be advantageous, we argue that this will rarely be the case in practice. Thus if waiting would be necessary to fulfill the start time t^{start} of a customer we must do so before starting the fragment as a whole. To acquire the earliest starting time of a fragment and the best relative timing of all stops within it, we only need to perform a backward pass followed by a forward pass over the fragment's stops, considering the respective pickup time windows and travel durations. Similarly, we can calculate the latest starting time for the fragment to avoid any lateness or the minimum total lateness when lateness cannot be avoided.

4.4 On-The-Fly Charging Stop Insertion

Due to the changes in the objective function and the disallowing of waiting within fragments in the considered problem variant, we cannot adopt the OTF charging stop insertion procedure from [5], but propose a new one tailored to our DynEADARP variant. It is a Dynamic Programming (DP) procedure that is guaranteed to insert charging stops (CSs) in an optimal way in respect to our objective function and a given route r of a vehicle.

First, any possibly already existing charging stops are removed; let n_r now be the number of fragments in the route. By $j = 1, \dots, n_r$ we denote the possible insertion positions of CSs in the route in front of each fragment.

Next, we identify for each possible insertion position a charging station s_j that induces the fastest detour. We only need to consider these charging stations for insertion, as selecting any other station would induce longer travel times.

The aim is now to determine a subset of these CSs to be inserted so that (a) the route becomes feasible and (b) the route's contribution to the objective function (1) consisting of the total travel duration plus the scaled total lateness is minimized. This is achieved as follows.

- Let $X \subseteq \{1, \dots, n_r\}$ be the charging stops to be realized in a route.
- We calculate/store for each position $j = 0, \dots, n_r$ states containing:
 - t : time when starting fragment j for $j > 0$ or t^{avail} for $j = 0$
 - dist : total distance traveled when arriving at fragment j for $j > 0$
 - d^{travel} : total travel time when arriving at fragment j for $j > 0$
 - B : battery level when arriving at fragment j for $j > 0$ or $B_{k,0}$ for $j = 0$
 - d^{late} : so far aggregated lateness
 - x : flag indicating if charging stop is inserted before fragment j for $j > 0$
 - prev : reference to predecessor state
- Let the initial state for $j = 0$ be $(t_k^{\text{avail}}, 0, 0, B_{k,0}, 0, \text{false}, \text{none})$
- The states for j are derived from all states for $j - 1$ by considering the two cases of either inserting the charging stop ($x = 1$) or not ($x = 0$), whereby the vehicle must not run out of battery. Moreover, a dominance check is performed among any two states at each level and dominated states are discarded. State p_1 dominates state p_2 iff

$$t(p_1) \leq t(p_2) \wedge d_{p_1}^{\text{travel}} \leq d_{p_2}^{\text{travel}} \wedge B(p_1) \geq B(p_2) \wedge d_{p_1}^{\text{late}} \leq d_{p_2}^{\text{late}}. \quad (2)$$

4.5 A Simple Heuristic to Counteract Pure Myopic Behavior

In preliminary experiments with the LNS as described so far, we soon recognized a significant weakness in the dynamic context: Due to the pure myopic view of the LNS and its OTF CS insertion, charging stops are frequently postponed as far as possible. In particular, when charging is not necessary for a vehicle with the currently assigned requests, charging will not be done at all, even when the vehicle is very low on charge and the vehicle is just waiting at the end of its route. Charging may then become immediately necessary when new requests arrive and possibly induce substantial lateness.

To counteract this behavior, we propose a simple heuristic strategy: We extend the objective function used by the DP for the OTF CS insertion by an additional term that corresponds to the time needed to fully charge the battery at the current state (without considering the travel time to a CS) and weigh this time by a tunable factor $\gamma \geq 0$. In addition to the DP above, we consider a variant of it in which the objective of the state p in DP is

$$f(p) = d_p^{\text{travel}} + \rho \cdot d_p^{\text{late}} + \gamma \cdot \frac{Q - B(p)}{\alpha}. \quad (3)$$

In this way, the charging station insertion is nudged towards charging EVs earlier.

When considering this extension, it may be an improvement to charge also at the end of a current route, and we therefore consider this possibility. The charging station insertion positions are therefore indexed by $j = 1, \dots, n_r + 1$ now, where position $n_r + 1$ indicates the end of the route.

Finally, to set the actual starting times of the fragments and possibly inserted charging stops in the route, we only need to perform a simple forward pass over the fragments and charging stops, considering the travel durations and earliest start times. Alternatively, we may also consider the latest possible start times that do not yield a higher objective value; these can be derived via an additional backward pass considering the ends of the time windows of pickup stops. While in the dynamic context latest possible start times may have the advantage of possibly more efficient future routes when new requests arrive, they also bear the danger of leading to higher lateness in the future. We therefore resort to the more conservative earliest possible start times as our standard strategy.

5 Guiding the LNS with Reinforcement Learning

Concerning the heuristic extension above to incentivize charging, the question remains how to determine a suitable value for the parameter γ . In our experiments, we performed tuning using *irace* [19] on representative instances to identify a reasonable, robust constant value, which turned out to be 0.025. As an advanced alternative, we propose to use Reinforcement Learning (RL) for learning a policy to select a value for γ dependent on features from the problem instance and current state. Moreover, we also use this approach to decide dynamically depending on the features whether fragments should be started at their earliest or latest possible times concerning the same minimal objective values. This learning is done in an offline manner on representative training instances, and the learned policy is then applied for actual benchmarking.

As RL method, we use Proximal Policy Optimization (PPO) [30], with the reward defined as the negative contribution of the *subsol* to the overall *sol* and one training step being generated each call to the *update* function in algorithm 1. We considered a variety of features as input for the policy to be learned but finally decided to use only the following ones that appeared to be most relevant:

- NEXP1, NEXP2, NEXP3: expected request arrival rates of the next three hours
- NCUR: number of currently known requests not yet served
- LATE: total lateness of so far planned routes for the known unserved requests

To make these features less dependent on the actual problem size, we normalize them all by dividing by the number of vehicles n_K . Concerning the output of the policy, the decision whether to start fragments as early or as late as possible is binary. Instead of letting the policy decide on arbitrary fractional values for γ , we found it more effective to turn this into a categorization task on the small set of discrete values $\{0, 0.025, 0.5\}$.

6 Experimental Results

We implemented our algorithms in Julia 1.11.4. For the PPO, we used Python 3.12.3 with the `stable-baselines3` package in version 2.6.0, configured to use a multi-layer perceptron; default parameter values were used here in all aspects except the entropy coefficient, which was set to 0.01 to encourage exploration due to the stochastic nature of the problem. All results were obtained on a single core of an AMD EPYC 7402 processor.

We use the benchmark set from [15], which consists of 30 randomly generated euclidean instances for each of 10, 20, and 50 requests, two vehicles, and a time horizon of 24h; requests arrive according to a homogeneous Poisson process. As these instances are quite small, we extended this benchmark set substantially with instances having 100, 200, 500, 1000, 2000, 5000, and 10000 requests and up to 355 vehicles. Due to the smaller average distance between adjacent requests in larger instances, we reduced the relative number of vehicles so that charging is still required for most vehicles; for details, see Table 1. The procedure used to create these instances is otherwise the same as in [15], and we therefore refer to this work for more details. We refer to all these instances as **uniform** instances.

Moreover, we created corresponding instances with inhomogeneous request arrival rates. For this purpose, we derived a probability density function for request arrivals from the New York City taxi data set [22] over all weekdays from June 2024; see Fig. 1. We call this benchmark set **nyctaxi**. All our benchmark instances are available online¹.

We compare our pure myopic LNS, the LNS approach with the heuristic incentive for charging (LNS+heu), and the RL-guided LNS (RL-LNS) on all these benchmark sets and the genetic programming hyperheuristic (GP-HH)

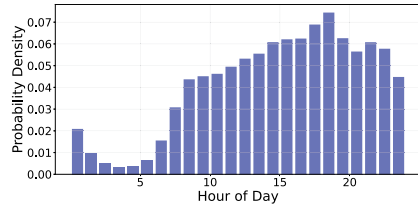


Fig. 1. NYC taxi request arrival distribution over all weekdays in 06/2024.

¹ <https://www.ac.tuwien.ac.at/files/resources/instances/DynEADARP/instances.zip>.

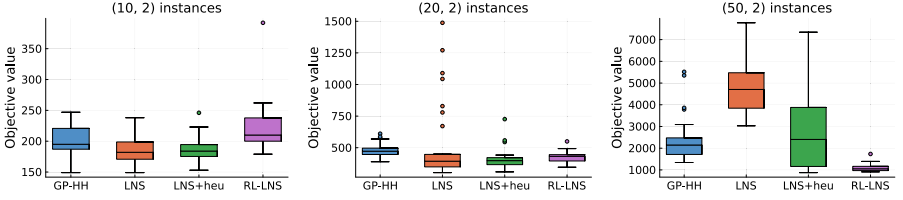


Fig. 2. Obtained objective values on the small benchmark instances of [15].

Table 1. Average %-gaps to best known solutions on **uniform** instances.

size (n, n_k)	best sol.	GP-HH		LNS		LNS+heu		RL-LNS	
	obj. val.	%-gap	σ	%-gap	σ	%-gap	σ	%-gap	σ
(10, 2)	183	8.976	7.584	0.539	1.032	2.315	2.608	21.487	20.307
(20, 2)	385	25.133	15.187	34.248	71.487	6.097	11.328	10.864	4.947
(50, 2)	1077	122.961	92.016	352.331	92.799	144.611	144.636	1.649	4.166
(100, 4)	1693	–	–	439.312	103.232	129.136	152.362	0.844	2.307
(200, 8)	2960	–	–	424.340	114.235	56.704	75.114	0.875	2.084
(500, 20)	6177	–	–	425.372	85.565	19.998	51.691	4.959	3.975
(1000, 40)	10903	–	–	345.759	70.465	0.929	3.049	8.547	3.392
(2000, 78)	19946	–	–	286.779	80.872	0.000	0.0	10.829	1.331
(5000, 180)	46422	–	–	323.117	51.928	0.000	0.0	10.119	0.715
(10000, 355)	89931	–	–	241.815	47.723	0.000	0.0	10.791	0.556

from [15] using the results from this work on the small homogeneous instances. For each of the 30 instances of each size and each benchmark set, 30 independent runs were performed. Each LNS run in an iteration of the dynamic framework was stopped after five seconds or when the incumbent solution did not improve over the last 300 iterations. The size of the random destroy operator in the LNS was tuned to 3 using irace [19] on *nyctaxi* instances with sizes between 50 and 1000. The destroy/repair operator combination used in each iteration was selected uniformly at random.

For RL-LNS, the policy training with PPO was done over 4000 randomly generated *nyctaxi* training instances with 50 requests and two vehicles. In our experiments, we therefore also test the generalization capability of the learned policy to substantially larger instances.

Figure 2 shows boxplots for the obtained objective values of the methods on the small **uniform** instances from [15]. They reveal substantial differences in solution quality among the approaches. On the smallest 10-request instances, all approaches performed relatively similarly with the LNS and LNS+heu having slightly smaller objective values than GP-HH. On the larger 50 request instances, the weaknesses of the myopic LNS become apparent, as here reasonable charging decisions are crucial and the myopic LNS runs into substantial lateness. RL-LNS dominates this size, also substantially outperforming GP-HH, which yields

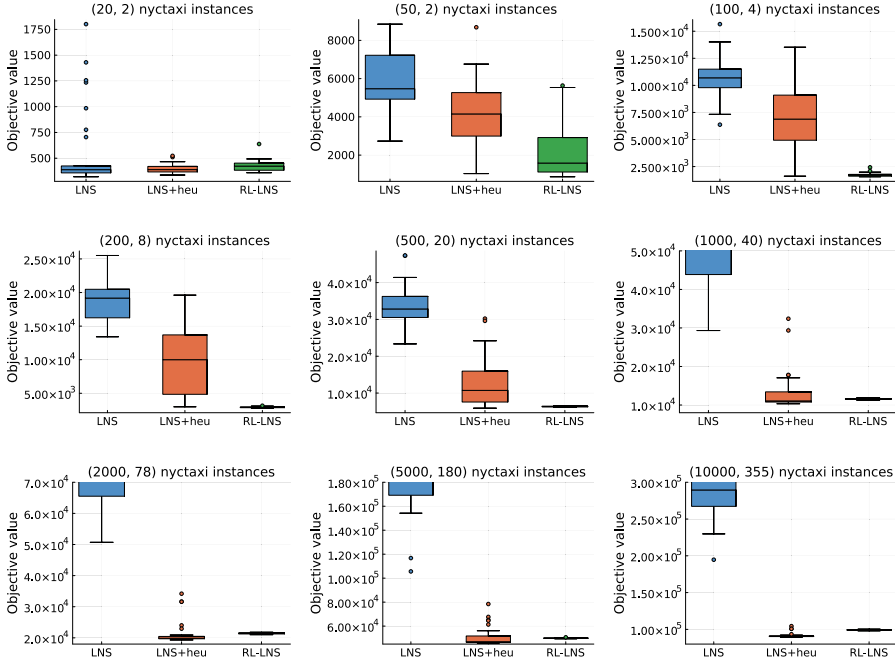


Fig. 3. Obtained objective values on *nyctaxi* instances with 20 to 10000 requests.

objective values that are already almost twice as large as those of RL-LNS. This suggests that GP-HH does not scale well to larger instances.

Table 1 shows average objective values of overall best solutions for each instance size of our extended *uniform* benchmark instance set and average %-gaps with respect to these best objective values together with corresponding standard deviations σ for each solving approach. The best results are highlighted in bold. When looking at the larger instances, in general differences among RL-LNS and LNS+heu become smaller. This can be attributed to the reduced generalizability of the model as the instance size increases. Note also that for the largest 5000 and 10000 request instances, the bottleneck is the time limit of five seconds instead of the iteration limit.

Next, we examine results on the *nyctaxi* instances with their inhomogeneous request arrival rates, for which Fig. 3 shows boxplots. For clarity, the box depicting the LNS results is cut off for instance sizes 1000 and above to emphasize the difference between LNS+heu and RL-LNS. The complete results and average %-gaps with respect to the overall best-obtained solutions are listed in Table 2. On average, RL-LNS performs best on instances ranging from 50 to 1000 requests. The model-based policy demonstrates strong generalization to instances up to 10 times larger than those seen during training substantially outperforming other approaches on instances of comparable size to the training data. Additionally, RL-LNS exhibits a low standard deviation across instances of varying sizes and types, indicating consistently stable performance.

Table 2. Average %-gaps to overall best known solutions on inhomogeneous **nyctaxi** instances.

size (n, n_k)	best sol. obj. val.	LNS		LNS+heu		RL-LNS	
		%-gap	σ	%-gap	σ	%-gap	σ
(10, 2)	193	0.214	0.733	2.495	2.592	14.764	5.934
(20, 2)	386	39.190	78.362	3.292	6.448	10.788	7.142
(50, 2)	2123	242.329	159.492	126.806	118.417	1.682	6.401
(100, 4)	1743	516.511	104.532	297.217	147.211	0.006	0.034
(200, 8)	2915	536.321	96.614	233.660	172.322	0.000	0.000
(500, 20)	6276	441.737	80.484	100.868	104.050	0.970	2.280
(1000, 40)	11069	351.630	80.758	20.430	44.240	4.705	3.963
(2000, 78)	20189	270.924	72.207	5.566	15.378	6.282	3.472
(5000, 180)	47887	291.907	63.687	6.791	14.089	4.325	3.325
(10000, 355)	91574	215.222	41.423	0.289	1.146	8.455	2.993

7 Conclusions and Future Work

In this work, we proposed an LNS-based approach utilizing an effective, exact on-the-fly charging stop insertion procedure for the dynamic electric autonomous dial-a-ride problem. Recognizing a major weakness of the pure myopic LNS concerning too late charging in the dynamic context, we extended the charging stop insertion by applying a heuristic to encourage earlier charging. This extension turns out to be crucial and highly effective particularly on medium-sized instances to obtain good solution quality. A more sophisticated enhancement is to learn in an offline manner by reinforcement learning—in our case proximal policy optimization—a policy to guide the LNS in the dynamic scenario. This is done by using selected features of the current state together with expected request arrival rates of the next few hours and controlling whether fragments should start earlier or later and how strong the incentive for charging should be. This RL-based LNS approach showed significant advantages, especially on instances in the range of 50 to 500 requests, but did not always dominate on the larger instances; the simpler approach with a constant factor for the heuristic incentive of charging was more effective on the larger instances where the model based policy did no longer generalize as well. The former genetic programming hyperheuristic could not compete well already on instances with 20 and 50 requests, and cannot be expected to scale reasonably to much larger instances.

Future research avenues include diverse refinements of the LNS, which besides the new OTF CS insertion procedure, is relatively basic. Moreover, the investigation of other data-driven methods for controlling the LNS or increasing its effectiveness in the dynamic context appears highly promising. Although the

existing GP-HH was clearly outperformed, genetic programming may still be a promising alternative to learn a policy for guiding a strong optimization technique like our LNS in a dynamic context.

References

1. Bongiovanni, C., Geroliminis, N., Kaspi, M.: A ride time-oriented scheduling algorithm for dial-a-ride problems. *Comput. Oper. Res.* **165**, 106588 (2024)
2. Bongiovanni, C., Kaspi, M., Cordeau, J.F., Geroliminis, N.: A machine learning-driven two-phase metaheuristic for autonomous ridesharing operations. *Transp. Res. Part E* **165**, 102835 (2022)
3. Bongiovanni, C., Kaspi, M., Geroliminis, N.: A two phase heuristic approach for the dynamic electric autonomous dial-a-ride problem. In: 7th Symposium of the European Association for Research in Transportation (2018)
4. Bongiovanni, C., Kaspi, M., Geroliminis, N.: The electric autonomous dial-a-ride problem. *Transp. Res. Part B* **122**, 436–456 (2019)
5. Bresich, M., Raidl, G.R., Limmer, S.: Letting a large neighborhood search for an electric dial-a-ride problem fly: on-the-fly charging station insertion. In: *Proceedings of the Genetic and Evolutionary Computation Conference*, pp. 142–150. GECCO '24, ACM (2024)
6. Cordeau, J.F., Laporte, G.: The dial-a-ride problem: models and algorithms. *Ann. Oper. Res.* **153**(1), 29–46 (2007)
7. Cordeau, J.F.: A branch-and-cut algorithm for the dial-a-ride problem. *Oper. Res.* **54**(3), 573–586 (2006)
8. Cordeau, J.F., Laporte, G.: A tabu search heuristic for the static multi-vehicle dial-a-ride problem. *Transp. Res. Part B* **37**(6), 579–594 (2003)
9. Coslovich, L., Pesenti, R., Ukovich, W.: A two-phase insertion technique of unexpected customers for a dynamic dial-a-ride problem. *Eur. J. Oper. Res.* **175**(3), 1605–1615 (2006)
10. Cubillos, C., Urrea, E., Rodríguez, N.: Application of genetic algorithms for the DARPTW problem. *Int. J. Comput. Commun. Control* **4**(2) (2009)
11. Garaix, T., Artigues, C., Feillet, D., Josselin, D.: Optimization of occupancy rate in dial-a-ride problems via linear fractional column generation. *Comput. Oper. Res.* **38**(10), 1435–1442 (2011)
12. Gaul, D., Klamroth, K., Stiglmayr, M.: Solving the dynamic dial-a-ride problem using a rolling-horizon event-based graph. In: *21st Symposium on Algorithmic Approaches for Transportation Modelling, Optimization, and Systems (ATMOS 2021)*, pp. 8:1–8:16. Schloss Dagstuhl, Leibniz-Zentrum für Informatik (2021)
13. Heitmann, R.J.O., Soeffker, N., Klawonn, F., Ulmer, M.W., Mattfeld, D.C.: Accelerating value function approximations for dynamic dial-a-ride problems via dimensionality reductions. *Comput. Oper. Res.* **167**, 106639 (2024)
14. Ho, S.C., Szeto, W., Kuo, Y.H., Leung, J.M., Petering, M., Tou, T.W.: A survey of dial-a-ride problems: Literature review and recent developments. *Trans. Res. Part B* **111**(C), 395–421 (2018)
15. Huang, W., et al.: Genetic programming hyper-heuristic for the dynamic electric dial-a-ride problem. In: *IEEE Congress on Evolutionary Computation*. IEEE (2025)
16. Kullman, N.D., Cousineau, M., Goodson, J.C., Mendoza, J.E.: Dynamic ride-hailing with electric vehicles. *Transp. Sci.* **56**(3), 775–794 (2022)

17. Limmer, S.: Bilevel large neighborhood search for the electric autonomous dial-a-ride problem. *Trans. Res. Interdisc. Perspectives* **21**, 100876 (2023)
18. Liu, C., Quilliot, A., Toussaint, H., Feillet, D.: A guided insertion mechanism for solving the dynamic large-scale dial-a-ride problem. In: *Proceedings of the 11th International Network Optimization Conference (INOC)*, pp. 70–75 (2024)
19. López-Ibáñez, M., Dubois-Lacoste, J., Pérez Cáceres, L., Stützle, T., Birattari, M.: The irace package: iterated racing for automatic algorithm configuration. *Oper. Res. Perspectives* **3**, 43–58 (2016)
20. Molenbruch, Y., Braekers, K., Caris, A.: Typology and literature review for dial-a-ride problems. *Ann. Oper. Res.* **259**(1), 295–325 (2017)
21. Molenbruch, Y., Braekers, K., Caris, A., Vanden Berghe, G.: Multi-directional local search for a bi-objective dial-a-ride problem in patient transportation. *Comput. Oper. Res.* **77**, 58–71 (2017)
22. NYC-Government: New York City taxi dataset (2024). <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>
23. Pillac, V., Gendreau, M., Guéret, C., Medaglia, A.L.: A review of dynamic vehicle routing problems. *Eur. J. Oper. Res.* **225**(1), 1–11 (2013)
24. Pisinger, D., Ropke, S.: Large neighborhood search. In: Gendreau, M., Potvin, J.Y. (eds.) *Handbook of Metaheuristics*, pp. 399–419. Springer (2010)
25. Jorgensen, R.M., Bergvinsdottir, J.L., K.B.: Solving the dial-a-ride problem using genetic algorithms. *J. OR Soc.* **58**(10), 1321–1331 (2007)
26. Rist, Y., Forbes, M.A.: A new formulation for the dial-a-ride problem. *Transp. Sci.* **55**(5), 1113–1135 (2021)
27. Ritzinger, U., Puchinger, J., Hartl, R.F.: Dynamic programming based metaheuristics for the dial-a-ride problem. *Ann. Oper. Res.* **236**(2), 341–358 (2016)
28. Ropke, S., Cordeau, J.F., Laporte, G.: Models and branch-and-cut algorithms for pickup and delivery problems with time windows. *Networks* **49**(4), 258–272 (2007)
29. Sayarshad, H.R., Gao, H.: A scalable non-myopic dynamic dial-a-ride and pricing problem for competitive on-demand mobility systems. *Transp. Res. Part C* **91**, 192–208 (2018)
30. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms ([arXiv:1707.06347](https://arxiv.org/abs/1707.06347)) (2017)
31. Su, Y., Dupin, N., Puchinger, J.: A deterministic annealing local search for the electric autonomous dial-a-ride problem. *Eur. J. Oper. Res.* **309**(3), 1091–1111 (2023)
32. Tafreshian, A., Abdolmaleki, M., Masoud, N., Wang, H.: Proactive shuttle dispatching in large-scale dynamic dial-a-ride systems. *Transp. Res. Part B* **150**, 227–259 (2021)
33. Tomandl, L., Jatschka, T., Raidl, G., Rodemann, T.: A learning two level optimization approach for the demand maximizing battery swapping station location problem. In: *International Conference on Computer Aided Systems Theory*, pp. 251–262 (2024)
34. van Lon, R.R., Holvoet, T., Vanden Berghe, G., Wenseleers, T., Branke, J.: Evolutionary synthesis of multi-agent systems for dynamic dial-a-ride problems. In: *Proceedings of the 14th Annual Conference on Genetic and Evolutionary Computation (GECCO '12)*, pp. 331–336. ACM (2012)